

Assignment 4

Author: Lan Stare
Date: 25. 9. 2025

Question 1

Our starting vertex is 0. We first assign the distances for all values to infinity except for 0, which we set to 0. In the first iteration of the main loop, we add 1 and 2 to the frontier and remember the distances 30 and 40. We note 1 as explored and put 3 and 4 to the frontier. For each of: 2, 3, 4 we now check the distances to them from 0 and see that the distance from 0 to 2 is the shortest. We therefore note 2 as explored. On this step we don't put any new vertices into the frontier. We chose vertex 3 as our next explored, because the shortest distance from 0 to 3 is through 1 and is equal to 50. The shortest distance from 0 to 4 is through 1 and 3 which is $30 + 20 + 20 = 70$. We then note number 4 as explored since we didn't add any nex vertices from number 3. We then add vertex 5 to the frontier and in the next iteration finish the algorithm when we find the shortest distance from 0 to 5 which is through 1, 3 and 4 and is equal to 110.

Question 2

Let's assume we have nodes a, b and c with paths: (a, b) = 2, (a, c) = 4 and (c, b) = -3.

The algorithm will start at a, where it will discover b and c. Since (a, b) is shortest, it will explore b first. Then it will travel from a to c (since we have't added anything new to the frontier). We now have the distance to b noted as 2 and distance to c noted as 4. The algorithm stops here since it has already explored all vertices, eventhough the shortest distance from a to b is through c (and that is $4 - 3 = 1$).

Question 3

```
In [31]: import heapq
import math

example_graph2 = {
    'Madrid': {'Rome': 1, 'Amsterdam': 2, 'Berlin': 3},
    'Amsterdam': {'Madrid': 2, 'Berlin': 1},
    'Berlin': {'Amsterdam': 1, 'Madrid': 3, 'Rome': 2, 'Stockholm': 2, 'Kyiv': 3, 'Moscow': 6},
    'Rome': {'Madrid': 1, 'Berlin': 2},
    'Stockholm': {'Berlin': 2, 'Moscow': 3},
    'Kyiv': {'Berlin': 3, 'Moscow': 2},
    'Moscow': {'Berlin': 6, 'Stockholm': 3, 'Kyiv': 2}}

def easiest_path(graph, source_vertex):
    """Note to self: This is NOT just a normal Dijkstra's algorithm since the path will only change when there is a shorter distance and won't change when distances are the same. So it doesn't"""

    #Initialization
    length = {u: math.inf for u in graph} #this gives each city infinite length
    predecessor = {u: None for u in graph}
    flights = {u: math.inf for u in graph} #we will check the number of flights from source with this dict
    length[source_vertex] = 0
    flights[source_vertex] = 0

    priority_queue = [(0, 0, source_vertex)] #Dijkstras algorithm uses a priority queue :) (distance, flights, vertex)

    #Main Loop
    while priority_queue: #creating a Loop for each city if queue isnt empty
        current_distance, current_flights, current = heapq.heappop(priority_queue) #taking the first element of the queue, Like Q.popleft()
        # Skip if we already found a better path (shorter or equal but with fewer flights)
        if (current_distance > length[current]) or (
            current_distance == length[current] and current_flights > flights[current]
        ):
            continue

        for city, weight in graph[current].items(): #This way we can already name both things we need (the city and their distance to current)
            new_distance = current_distance + weight
            new_flights = current_flights + 1
            if (new_distance < length[city]) or (new_distance == length[city] and new_flights < flights[city]): #when the distance is shorter OR the same and Less flights
                length[city] = new_distance
                predecessor[city] = current
                flights[city] = new_flights
                heapq.heappush(priority_queue, (new_distance, new_flights, city))

    for city in length:
        print(f"- {city}: {length[city]} flights")

    return ""

print(easiest_path(example_graph2, 'Madrid'))

- 0 for Madrid.
- 1 for Amsterdam.
- 1 for Berlin.
- 1 for Rome.
- 1 for Stockholm.
- 2 for Kyiv.
- 3 for Moscow.
:)
```

Correctness: This is mostly just Dijkstra's algorithm which is known for finding the shortest path in weighted graphs (we discussed this in the lecture). We only added flight counting and 'tie breaking' meaning that if two flights have the same lenght, the algorithm picks the one that has less flights.

Efficiency of my algorithm stays the same: Each heapq.heappush and heapq.heappop takes $O(\log(|V|))$ time. Each vertex from graph is processed once so it happens in time $O(|V|)$. The comparison of distances OR flights runs once per edge making it $O(|E|)$. This makes the joint complexity: $O((|V| + |E|)\log(|V|))$.

Question 4

The idea is to replace the min-heap with arrays (this is sadly not my own idea but an idea I learnt from the internet).

```
In [35]: example_graph2 = {
    'A': {'B': 1, 'C': 2},
    'B': {'D': 1, 'E': 3},
    'C': {'E': 1},
    'D': {'E': 1},
    'E': {}
}

def dial_shortest_paths(graph, source, W):
    # Initialization
    dist = {v: float('inf') for v in graph}
    dist[source] = 0

    buckets = [[] for _ in range(W + 1)]
    buckets[0].append(source)

    current_index = 0

    while True:
        # Find next non-empty bucket
        while not buckets[current_index]:
            current_index = (current_index + 1) % (W + 1)
            if all(not b for b in buckets):
                return dist

        u = buckets[current_index].pop()

        for v, weight in graph[u].items():
            new_dist = dist[u] + weight
            if new_dist < dist[v]:
                dist[v] = new_dist
                bucket_index = new_dist % (W + 1)
                buckets[bucket_index].append(v)

    print(dial_shortest_paths(example_graph2, 'A', 5))

{'A': 0, 'B': 1, 'C': 2, 'D': 2, 'E': 3}
```

Correctness: Buckets ensure that when we pick a vertex u from buckets[current_index], no other vertex with a smaller tentative distance remains unprocessed. Relaxation step maintains the invariant that dist[v] is the length of the shortest path found so far. Because edge weights are non-negative, once we process u, its shortest distance is finalized (just like in standard Dijkstra).

Complexity: Initialization: Create dist: $O(|V|)$, Create buckets: $O(W)$

Main Loop: Each vertex is inserted into a bucket at most once per relaxation: $O(|V|)$ total bucket insertions. Each edge is relaxed at most once: $O(|E|)$ total.

Scanning buckets: we may advance the current_index at most W times per vertex because we never jump more than W ahead in distance.

Total cost of bucket scans: $O(W|V|)$

So total complexity: $T(V,E)=O(W|V|+|E|)$

Question 5

```
In [4]: from IPython.display import Image, display

# Replace with your file path
image_path = "4ex.jpeg"

display(Image(filename=image_path))
```

